Where Did It Go?

Product Requirements Document (v4.0) | React Native (Expo) + Supabase

This is the consolidated final PRD. It preserves the original feature scope, neobrutalist design direction, and all 11 build phases, while clarifying financial calculations, account isolation, recurring transactions, and offline synchronization. It supersedes all previous revisions.

1. Product Overview

A mobile app for tracking income and expenses, setting monthly savings goals, managing category budgets, and understanding monthly cash flow. The app is offline-first: screens read from a local SQLite database, and user changes are saved locally before synchronization. After initial sign-in and setup, users can record and review transactions without an internet connection. The Welcome screen acts as both the landing page and sign-in page. Returning users with an existing local session skip it.

1.1 Primary user outcomes

- Quickly record income and expenses.
- Trust that saved transactions persist offline.
- Understand monthly income, spending, and savings.
- Monitor category budgets and savings goals.
- Access synchronized data on multiple devices.
- Understand when changes are pending, failed, or conflicting.

1.2 Product priorities

1. Data correctness and durability.
2. Clear financial calculations.
3. Reliable offline use and synchronization.
4. Fast, accessible transaction entry.
5. Consistent visual design.
6. Charts, animations, and convenience features.

1.3 Offline boundaries

- First sign-in and initial account setup require internet access.
- Previously downloaded data remains usable offline.
- Local transaction entry, search, calculations, charts, goals, and parsing work offline.
- Remote changes and server-generated recurring transactions appear after synchronization.
- Authentication, account recovery, and synchronization require connectivity.

2. Technology Stack

Layer	Choice
App	React Native with Expo and TypeScript
Build environment	Expo development build
Navigation	Expo Router
Local database	`expo-sqlite`
Backend	Supabase Postgres
Authentication	Supabase Auth with Google and email/password
Client-generated IDs	`expo-crypto` UUIDs
Secure session storage	`expo-secure-store`
Natural-language parsing	Local regex parsing and `chrono-node`
Charts	`react-native-svg`-based bar and line charts
Long lists	FlashList
Animations	Reanimated
Release builds	EAS Build

Select compatible package versions during setup and record them in the repository. Keep database access, synchronization, calculations, and parsing separate from screen components.

3. Scope

3.1 Included in v1

- Google and email/password authentication.
- Income and expense creation, editing, and deletion (with Undo).
- Default and custom categories.
- Monthly income, spent, and saved summaries.
- Monthly savings goals and repeating monthly category budgets.
- Offline natural-language transaction entry.
- Bar and line charts. Server-generated recurring transactions.
- Transaction search, filters, and CSV export.
- Local SQLite persistence and multi-device synchronization.
- Visible sync status and conflict resolution.

3.2 Out of scope

Bank synchronization, receipt scanning, shared expenses or shared accounts, currency conversion or multi-currency bookkeeping, account balances, loans, net-worth tracking, and dark mode.

4. Screens and Navigation

Screen	Contents
Welcome / Sign-in	Logo, tagline, feature highlights, Google sign-in, email sign-in/sign-up, password recovery
Dashboard	Month selector, Income/Spent/Saved cards, deficit warning, savings-goal progress, charts, recent transactions, sync status
Add/Edit Transaction	Amount, income/expense type, category, date, note, natural-language input, parsed preview
Transactions	Search, date/category/type/amount filters, edit, delete, undo
Goals	Create, edit, and delete monthly savings goals; show progress against monthly savings
Insights	Monthly spending comparison, category breakdown, income-versus-expense bars, savings trend
Settings	Currency, categories, budgets, recurring rules, CSV export, sync details, conflict resolution, sign-out

Financial screens read exclusively from SQLite, not directly from remote queries.

5. Financial and Date Rules

5.1 Money representation

- Store monetary values as integers in the currency's smallest unit.
- Store positive transaction amounts; type determines income or expense.
- Store the account currency code and decimal precision.
- Parse decimal input using string-based conversion to integer units.
- Reject zero, negative, malformed, unsupported-precision, and out-of-range values.
- Keep stored amounts and calculated totals within JavaScript's safe integer range.
- Enforce matching validation on the server.

5.2 Currency

Choose one supported currency during onboarding. Currency selection applies to the entire account. v1 does not perform exchange-rate conversion. Currency may be changed only before any financial records exist.

5.3 Monthly calculations

For the selected calendar month:

```
Income = ∑ (non-deleted income transactions)
Spent = ∑ (non-deleted expense transactions)
Saved = Income − Spent
```

- Use transaction dates, not creation or synchronization timestamps. Include pending local changes.
- Exclude soft-deleted transactions.
- If Saved is negative, display "Spent X more than earned." Do not label negative monthly savings as debt.
- Explain that Saved is monthly cash flow, not an account balance.

5.4 Dates

Store transaction dates as YYYY-MM-DD. Store goal months as the first day of the relevant month. Store server audit timestamps in UTC. A time-zone change must not shift an existing transaction's calendar date. Resolve words such as "today" and "yesterday" using the device's local date when parsing.

6. Feature Requirements

6.1 Transactions

Users can create, edit, and soft-delete income and expenses. Required fields: Amount, Type, Category, Date. Deletion hides the transaction immediately, shows an undo toast for 5 seconds. Undo restores the record locally and queues the sync operation.

6.2 Categories

Provide default categories. Allow custom names, icons, and colors. A transaction must reference a category owned by the same user with a compatible type. Archive categories that have historical transactions rather than deleting them. Archived categories remain visible in history but cannot be selected for new entries.

6.3 Savings goals

```
Progress = max(0, Monthly Saved) / Target Amount
```

Clamp the visual progress bar between 0% and 100%. Show actual saved amount even when negative or above target. Goals are independent benchmarks against the same monthly savings.

6.4 Category budgets

Support one active repeating monthly budget per expense category. Warn at 80% and 100%. Above the limit, display the amount exceeded.

6.5 Natural-language entry

Parse locally (e.g., "coffee 4.50 yesterday"). Never save automatically from an ambiguous parse. Show an editable preview before saving. Parsing must not require a network service.

6.6 Charts and insights

```
Change % = [(Current Spending − Previous Spending) / Previous Spending] × 100
```

Compare selected month with the preceding calendar month. Label an ongoing month as "month to date."

6.7 Recurring transactions

Generate transactions on the server only. An offline-created rule starts server generation only after it syncs. Catch up missed due occurrences after scheduler interruptions. For a monthly rule anchored to the 31st, use the final day in shorter months without changing the original anchor.

6.8 CSV export

Export non-deleted local transactions for a selected date range. Format amounts using the account's decimal precision. Escape CSV fields and neutralize spreadsheet-formula prefixes in user-controlled text.

7. Data Model

7.1 Common server fields

Synchronized entities include: `id`, `user_id`, `version`, `created_at`, `updated_at`, `deleted_at`. Use client-generated UUIDs for user-created entity IDs. The server controls audit timestamps and version increments.

7.2 Server tables

Table	Additional or defining fields
profiles	`id`, `currency`, `currency_decimals`, `version`, `created_at`, `updated_at` (no independent user_id)
categories	`name`, `icon`, `color`, `type`, `archived_at`
transactions	`type`, `amount`, `category_id`, `note`, `date`, `nullable recurring_id`, `nullable occurrence_date`
goals	`name`, `target_amount`, `month`
budgets	`category_id`, `monthly_limit`
recurring	`type`, `amount`, `category_id`, `note`, `frequency`, `start_date`, `next_date`, `schedule_timezone`, `anchor_day`, `is_active`

7.3 Local-only metadata

Maintain local synchronized entity copies, durable mutation outbox, local revision numbers, per-account pull progress, sync errors, and unresolved conflicts.

8. Offline Synchronization Architecture

8.1 Local source of truth

Every user mutation must validate input, update local entity, increment local revision, and write outbox mutation **in one SQLite transaction.**

8.2 Push behavior

Process related records in dependency order. Use immutable mutation IDs for retry deduplication. Commit a server mutation and its acknowledgment atomically.

8.3 Conflict handling

Use optimistic concurrency with server versions. Reject stale versions rather than silently overwriting newer server data. Preserve both the local proposed change and the latest server record. Conflict choices: "Keep my change" or "Use synced version."

8.4 Pull protocol

Do not use the phone’s clock as a pull cursor. Commit downloaded changes and their associated pull progress in one local transaction.

8.5 Triggers and status

Attempt sync on launch, app resume, connectivity recovery, new mutations, and manual retry. Background execution is best effort.

9. Authentication, Security, and Account Isolation

Enable row-level security on user-facing tables. Maintain a separate local database or strict partition per account. Never replay one user’s outbox under another user’s session. If a session expires, keep existing local data usable but pause synchronization.

10. UI/UX and Accessibility

Neobrutalist wallet-app direction: Cream background, Black 2–3px borders, Hard offset shadows, One primary accent color. Green for positive savings, red for deficits. Space Grotesk font. Support larger text and screen readers. Ensure charts have a textual equivalent.

11. Build Phases

Phase 1: Setup and Design

Tasks:

- Create the Expo TypeScript project and Git repository.
- Configure development builds and navigation.
- Define folder boundaries and environment configuration.
- Establish theme tokens and fonts.
- Build Button, Card, Input, Chip, Toast, and status components.
- Record package versions and setup instructions.

Done when: the app runs on a real device and a style-demo screen demonstrates the component system.

Phase 2: Database Layer — Supabase

Tasks:

- Create tables, constraints, indexes, and migrations.
- Add server-controlled timestamps and versions.
- Add row-level security.
- Create idempotent category seeding.
- Define mutation deduplication and pull-protocol storage.

Done when: schema tests pass and two test users cannot access each other's records.

Phase 3: Backend and Auth

Tasks:

- Implement Google and email/password authentication.
- Configure verification, recovery, and secure sessions.
- Implement version-checked mutation endpoints.
- Implement recurring generation with occurrence uniqueness.
- Test ownership validation and privileged-function access.

Done when: authentication works on a real device and recurring generation can run repeatedly without duplicate occurrences.

Phase 4: Local Storage and Offline Infrastructure

Tasks:

- Create SQLite schemas and migrations.
- Build local repositories and atomic entity/outbox writes.
- Add account partitioning, local revisions, and conflict storage.
- Build a narrow end-to-end sync prototype.

Done when: a transaction created offline survives an app restart and reaches a second device after synchronization.

Phase 5: Frontend Integration

Tasks:

- Build all seven screens.
- Connect financial screens to local repositories.
- Implement calculations, categories, goals, and budgets.
- Add parsing, charts, insights, and CSV export.
- Add empty states, undo, and validation.

Done when: all locally supported features work without connectivity after setup.

Phase 6: Sync Engine Integration

Tasks:

- Complete push, pull, retries, and dependency handling.
- Implement version conflicts and resolution UI.
- Handle deletion and restoration.
- Integrate recurring downloads.
- Add launch, resume, connectivity, manual, and best-effort background triggers.

Done when: two devices converge after tested offline edits without silent overwrites or duplicate operations.

Phase 7: Testing

Tasks:

- Unit-test money conversion, calculations, parsing, goals, budgets, and scheduling.
- Test outbox retries, lost responses, and edits during upload.
- Test pagination, concurrent pull changes, and interrupted bootstrap.
- Test two-device edit/delete conflicts.
- Test expired sessions, wrong clocks, and account switching.
- Test cross-user reads and writes.
- Test recurring month-end behavior.
- Test 1,000+ transactions, accessibility, and CSV safety.

Done when: automated tests and the real-device checklist pass without unresolved data-loss or security defects.

Phase 8: Debugging and Optimization

Tasks:

- Fix defects from testing.
- Profile list rendering, database queries, and chart calculations.
- Reduce unnecessary rerenders.
- Tune indexes, pagination, and retries.
- Validate migration and recovery behavior.

Done when: common flows remain responsive, long lists scroll smoothly on the target device, and normal use has no known crash or data-loss issue.

Phase 9: Polish

Tasks:

- Add restrained animations and haptics.
- Finalize app icon and splash screen.
- Refine copy, spacing, empty states, and sync feedback.
- Complete accessibility and reduced-motion checks.

Done when: screens are visually consistent and important states are understandable without explanation.

Phase 10: Launch

Tasks:

- Create EAS release builds.
- Prepare privacy documentation and store disclosures.
- Complete distribution-specific account-management requirements.
- Test the release build on a real phone.
- Create README screenshots, architecture diagram, setup guide, and demo GIF.
- Document known limitations.

Done when: the release is installable, the release checklist passes, and the repository is portfolio-ready.

Phase 11: After Launch

Tasks:

- Collect usage feedback.
- Monitor errors without collecting sensitive financial content.
- Prioritize reliability fixes.
- Review actual use of goals, budgets, parsing, and recurrence.
- Evaluate future features separately.

Done when: a documented feedback-and-maintenance process exists. This phase continues after launch.

12. Definition of Done

The v1 release is complete when locally saved changes survive app restarts, core bookkeeping works offline, two devices synchronize through the defined conflict policy without silent overwrites, financial math perfectly follows rules, and no critical security or data-loss defects remain.